



DigiPass

Know Your Products!

DASH_GUI: A Technical Deep Dive



Architecture · Enrichment Engine · SHACL Generation

Audience: Developers, architects, semantic web practitioners

45–60 minutes

Know Your Products!

COORDINATION AND KNOWLEDGE SHARING ACROSS MATERIALS DEVELOPMENT COMMUNITIES (CSA):
DigiPass CSA project has been funded by the European Commission for the programme HORIZON-CL4-2023-RESILIENCE-01, Grant Agreement No 101138510
WIKKI LIMITED, UK participant in Horizon Europe Project DigiPass, is supported by UKRI grant number 10100819: DigiPass





What We'll Cover

- 1 The Brick Approach** Semantic component reuse
- 2 Architecture Overview** Multi-tenant, shared core
- 3 The Enrichment Engine** 4-layer resolution — generic, not QUDT-only
- 4 SHACL Export Pipeline** From bricks to validation rules
- 5 Web vs Qt Frontends** Same core, different UI technology
- 6 Extension Points** Add ontologies, rules, templates





Why Traditional SHACL Authoring Fails

Traditional Approach

- Write raw Turtle by hand
- Manual unit lists (copy-paste)
- No reuse across schemas
- Validation errors found late
- Ontology experts only

Brick Approach

- Compose visual bricks
- Ontology-driven widgets (auto)
- Reuse across all schemas
- Enrichment hints while authoring
- Domain experts can author



The Brick Approach



What Is a Brick?

A Brick = NodeShape + PropertyShapes, stored as JSON, exported as SHACL.

```
{
  "name": "Mass",
  "object_type": "NodeShape",
  "target_class":
  "https://schema.org/QuantitativeValue",
  "leaf_properties": [{
    "path":
    "http://qudt.org/schema/qudt/value",
    "datatype": "xsd:decimal"
  }, {
    "path":
    "http://qudt.org/schema/qudt/hasUnit",
    "sh_class":
    "http://qudt.org/vocab/quantitykind/Mass",
    "min_count": 1
  }]
}
```

sh_class on the unit
property

→ triggers enrichment (QUDT lookup)
→ auto-populates unit dropdown

min/maxCount

cardinality constraints

JSON storage

fast CRUD, git-friendly

Export

→ SHACL Turtle + dash:editor
annotations



Architecture



Frontend Layer

Web (React/Babel) :5001 · Qt (PyQt6) desktop

REST API

Flask — multi-tenant, per-session BrickCore

Core Services

OntologyManager · EnrichmentEngine · SHACLExporter

Shared Library

JSON bricks/schemas · TTL ontology cache

Standards Out

SHACL Turtle · DASH editors · HTML forms

Both Web and Qt share the same BrickCore — no logic duplication.



The Shared Library System



```
ShaclForm-library/  
├── bricks/  
│   └── battery_dpp/  
│       ├── mass_b1000003-ba77-4000-8000-000000000003.json  
│       ├── temperature_b1000005-ba77-4000-8000-000000000005.json  
│       └──  
batterycapacity_b1000004-ba77-4000-8000-000000000004.json  
├── ...  
├── schemas/  
│   └── battery_dpp/  
│       └── ev-battery-dpp-001.json  
├── ontologies/  
│   └── cache/  
│       ├── qudt-units.ttl          (2906 units)  
│       ├── qudt_quantitykind.ttl  (1217 kinds)  
│       ├── battery_substances.ttl (25 substances, PubChem IRIs)  
│       └── ...
```

→ Filesystem-based — git-friendly, no database

→ Single source of truth for both Web and Qt

→ 17 ontologies loaded at startup (~45 MB TTL)

→ Add ontology → drop .ttl in cache → restart

→ New prefixes & enrichment rules available instantly



The Enrichment Engine



Layer 0: Datatype → Widget (fast path)

sh:datatype	widget type	rendered as
xsd:boolean	boolean_toggle	checkbox
xsd:date	date_picker	calendar widget
xsd:decimal	decimal_input	numeric field
xsd:anyURI	uri_input	URL-validated field
rdf:langString	language_text	multilingual input
xsd:integer	decimal_input	integer-constrained

No ontology lookup — purely declarative rules in widget_rules.ttl. Add new datatypes by adding TTL triples.



The Enrichment Engine



Layers 1-2: Semantic Resolution (QUDT is one example)

Physical quantities (QUDT)

```
sh:class = qudt:Mass
↓ Layer 1: dimensional sig
(1, 0, 0, 0, 0, 0, 0) [M1]
↓ Layer 2: applicableUnit
query QUDT graph

→ unit_dropdown
[kg, g, lb, oz, mg, t]
```

Controlled vocab (SKOS)

```
sh:class = skos:Concept
+ skos:inScheme <myVocab>
↓ Layer 2: extract concepts
from scheme graph

→ skos_selector dropdown
[Concept A, B, C ...]
(your vocabulary, not QUDT)
```

Any ontology with the right predicate structure works — rules declared in `widget_rules.ttl`, zero Python changes



The Enrichment Engine



Layers 3-4: Namespace & Inheritance Fallbacks

Layer	Trigger	Result	Example
3	Namespace prefix match	Suggest known properties	foaf:Person → name, mbox...
3	Predicate in loaded onto	Suggest known properties	schema:Thing → identifier...
4	rdfs:subClassOf chain	Entity search-as-you-type	foaf:Person $\sqsubseteq$ foaf:Agent
—	No match at any layer	Plain text field (fallback)	unknown:Widget → text input

Load any OWL/RDFS ontology → its classes and subclass chains become immediately available.

No vocabulary names are hardcoded in Python — purely data-driven.





widget_rules.ttl — Rules Without Code Changes

```
# Physical quantity — dimensional signature
dashgui:MassRule a dashgui:WidgetRule ;
    dashgui:widget          "unit_dropdown" ;
    dashgui:dimensionalSignature "1 0 0 0 0 0 0" ;
    dashgui:siUnit <qudt-unit:KiloGM> .

# Controlled vocabulary — predicate trigger
dashgui:SkosRule a dashgui:WidgetRule ;
    dashgui:widget          "skos_selector" ;
    dashgui:triggerPredicate skos:inScheme .

# Entity hierarchy — subClassOf trigger
dashgui:AgentRule a dashgui:WidgetRule ;
    dashgui:widget          "entity_lookup" ;
    dashgui:triggerSubClassOf foaf:Agent .
```

- ✓ No ontology names in Python
- ✓ 12 dimensional signatures
- ✓ 4 predicate-trigger rules
- ✓ 3 subClassOf-trigger rules
- ✓ Add your rule → add TTL triples





From Bricks to SHACL Turtle

```
# Brick (JSON)
```

```
{
  "name": "Mass",
  "object_type": "NodeShape",
  "target_class":
    "https://schema.org/QuantitativeValue",
  "leaf_properties": [{
    "path":
      "http://qudt.org/schema/qudt/value",
    "datatype": "xsd:decimal",
    "min_count": 1
  }, {
    "path":
      "http://qudt.org/schema/qudt/hasUnit",
    "sh_class":
      "http://qudt.org/vocab/quantitykind/Mass",
    "min_count": 1
  }]
}
```

export_schema()

```
# SHACL (Turtle)
schema:Mass a sh:NodeShape ;
  sh:targetClass ex:Mass ;
  sh:property [
    sh:path ex:value ;
    sh:datatype xsd:decimal ;
    sh:class qudt:Mass ;
    dash:editor
      dash:InstancesSelectEditor ;
    sh:in (unit:KG unit:G
          unit:LB unit:OZ) ;
  ] .
```

EnrichmentEngine runs at export time — dash:editor and sh:in resolved from sh:class via ontology lookup.





Widget Types → DASH Editors

widget type	dash:editor	dash:viewer
text	dash:TextFieldEditor	dash:LabelViewer
boolean_toggle	dash:BooleanSelectEditor	dash:BooleanViewer
date_picker	dash:DatePickerEditor	dash:LabelViewer
decimal_input	dash:DecimalFieldEditor	dash:LabelViewer
unit_dropdown	dash:InstancesSelectEditor	dash:LabelViewer
skos_selector	dash:InstancesSelectEditor	dash:LabelViewer
entity_lookup	dash:AutoCompleteEditor	dash:LabelViewer

DASH (Data Shapes) provides the form widgets — SHACL provides the validation constraints.





Web Frontend: React + Babel (single file)

```
index.html (inline JSX, Babel standalone — no build step)
├── App
│   ├── SessionManager
│   ├── BrickList (sidebar)
│   └── BrickEditor
│       ├── BasicFields
│       ├── LeafPropertiesList
│       └── PropertyEditorModal
│           ├── DatatypeSelector
│           ├── OntologyBrowser (all 17 ontologies)
│           └── EnrichmentDisplay (live widget preview)
```

REST API

Flask backend,
per-session BrickCore

Enrichment

useEffect hooks call
/api/enrichment

Ontology

Browser opens on 'All
Ontologies' merged
view

No build

Babel transpiles
in-browser — rapid
iteration

Future

Migrate to Vite + React
for production DX





Qt Frontend: PyQt6 + UI Files

```
brick_editor.py (main window)
├── loadUi('property_editor.ui') ← Qt Designer
│   ├── Constraint group boxes (dynamic show/hide)
│   ├── Datatype combo → _update_group_visibility()
│   └── Class browser button → SimpleOntologyBrowser
├── gui_components.py
│   ├── PropertyEditorDialog
│   ├── ConstraintEditorDialog
│   └── SimpleOntologyBrowser
└── brick_core_simple.py ← same as web backend
```

UI files	.ui XML — layout changes without recompile
loadUi()	Dynamic load at runtime — no pyuic step
Same core	BrickCore shared with Flask web API
Parity	Same constraint fields as web modal

Key Point: Same BrickCore, different UI technology — no logic duplication.



Extension Points



How to Extend the System

What	How	Result
Add ontology	Drop .ttl in ontologies/cache/	New classes & prefixes available
Add widget rule	Add triples to widget_rules.ttl	New trigger → new widget, no Python
Add datatype	Add rule to widget_rules.ttl	New datatype → new widget type
Custom validator	Subclass BrickCore	Hook into save/load pipeline
Custom exporter	Subclass SHACLEExporter	Custom Turtle templates or formats

Everything is pluggable — no ontology names hardcoded in Python.





Numbers

Ontologies loaded at startup	17 ($\approx$ 45 MB TTL)
QUDT units indexed	2,906
Quantity kinds indexed	1,217
Schema.org classes	2,987
Enrichment cache hit rate	$\sim$ 95 % (in-memory per session)
SHACL export time	< 100 ms per brick
Concurrent sessions tested	50+
Session memory footprint	$\sim$ 10 MB
Test suite	60 pytest tests, all passing



Key Takeaways



Bricks

Semantic, reusable SHACL components — NodeShape + PropertyShapes as JSON



Enrichment

4-layer resolution — generic, not QUDT-only; any ontology works



Multi-tenant

Web + Qt + API share one BrickCore — no logic duplication



Extensible

Add ontologies or TTL rules — zero Python changes required



Standards

SHACL + DASH output — works with any validator, no vendor lock-in



Questions



Q: Why not JSON Schema?

A: SHACL validates semantics (units, linked data); JSON Schema validates structure. Can convert.

Q: Why not use SPARQL for ontology queries?

A: rdflib supports it, but direct traversal is faster, simpler to debug, and sufficient — SPARQL adds value only for complex cross-graph joins.

Q: Can it work with property graphs (Neo4j)?

A: SHACL is RDF-native; but SHACL validates before storage — export to JSON/JSON-LD after.

Q: Why single-file React?

A: Rapid prototyping, no build step. Production: migrate to Vite + React bundle.

